UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECP3133-01 High Performance Data Processing

Project 2: Real Time Sentiment Analysis using Apache Spark and Kafka

LECTURER: ASSOC. PROF. MOHD SHAHIZAN BIN OTHMAN

NO.	NAME	MATRIC NUMBER
1.	IMAN ABADI BINN MOHD NIZWAN	A23CS0084
2.	MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112

Table of Contents

1.0 Introduction....3
1.1 Background....3
1.2 Objective....3
1.3 Scope....4
2.0 Data Acquisition and Preprocessing....4
2.1 Data Source....4
2.2 Preprocessing....5
2.3 Sentiment Labelling....6
3.0 Sentiment Model Development....8
3.1 Data Splitting....8
3.2 Handling Class Imbalance....8
3.3 Models....9
3.4 Evaluation and Comparison....9
4.0 Apache System Architecture....11
4.1 Data Preparation and Model Training....12
4.2 Real-Time Streaming....12
5.0 Analysis and Result....13
5.1 Sentiment Distribution....13
5.2 Dashboard....14
6.0 Optimization and Comparison....15
6.1 Model Comparison....15
6.2 Batch vs Streaming Comparison....15
7.0 Conclusion and Future Work....16
8. References....16

1.0 Introduction

1.1 Background

Organizations are increasingly depending on real-time analytics to evaluate public feedback about their products. Banking and e-wallet mobile applications in Malaysia receive a constant supply of user reviews, which directly show customer satisfaction with services such as payments, transfers and reliability. Analyzing this feedback allows the bank and e-wallet companies to detect issues and respond to it quickly.

This project implements a real-time sentiment analysis system that collects Malaysian app reviews. The collected reviews are then classified as positive, negative and neutral using trained machine learning models and processed through a streaming pipeline that was built by Apache Spark and Kafka. The classified results are then stored in ElasticSearch and visualized through interactive dashboards in Kibana.

1.2 Objective

1. Collect a volume of Malaysian e-wallet and banking app reviews from the Google Play Store.
2. Apply NLP preprocessing to prepare the text and use a pre-trained model for text-based sentiment label.
3. Train and compare at least two sentiment-classification models.
4. Handle the class imbalance present in the dataset.
5. Build a real-time pipeline using Apache Kafka and Apache Spark.
6. Store results in Elasticsearch and visualise them in Kibana.
7. Compare system performance between batch mode and streaming mode.

1.3 Scope

The dataset contains 16,882 reviews of nine Malaysian banking and e-wallet apps. Two models are compared which are Logistic Regression and a bidirectional LSTM. The best model from evaluation from model training is deployed in the streaming pipeline.

2.0 Data Acquisition and Preprocessing

2.1 Data Source

The dataset used for this project is user reviews of e-wallet apps from Google Play Store that is based in Malaysia. The reviews were collected using the *google-play-scraper* Python library. A collection script called collect.py was written to pull reviews for eight Malaysia e-wallet and banking applications shown in the table below showing the application name and the play store package ID used shown in Table 2.1.

The python script was configured for reviews strictly from Malaysia. The restrictions are applied in the python script where (country = “my”, language = “en”) and sorted to only take the “Most Relevant” reviews of up to 2,000 reviews per app. The result returned for each row has the review ID, user name, star rating, timestamp, review text, and “thumbs-up” count column. A total of 18,000 reviews were collected and saved into reviews_raw.csv file.

Table 2.1: Application List Used for Scraping

Application	Play Store Package ID	Reviews
Touch ’n Go eWallet	my.com.tngdigital.ewallet	2,000
Maybank MAE	com.maybank2u.life	2,000
Boost eWallet	my.com.myboost	2,000

ShopeePay (Shopee MY)	com.shopee.my	2,000
BigPay	com.tpaay.bigpay	2,000
CIMB OCTO MY	com.cimb.cimbocto	2,000
RHB Banking App	com.rhbgroup.rhbmobilebanking	2,000
Hong Leong Connect	my.com.hongleongconnect.mobileconnect	2,000
MyPB by Public Bank	com.pbb.mypb	2,000
Total	9 applications	18,000

2.2 Preprocessing

Text Cleaning

A function called clean_text() was made to apply all of the necessary steps on for text cleaning for sentiment analysis while still keeping the original text column. The cleaning steps done were:

- Lower case all text.
- Converting any emojis into sentiments. For example, an angry face emoji is translated as an angry sentiment.
- Removal of URLs, e-mail addresses, and @mentions. For hashtags, the text itself is kept but the symbol '#' is removed entirely.
- Expansion of Malay/Manglish contractions and shorthand using a pre-defined dictionary. For example, the word "tak" is replaced with "tidak".
- Normalization of elongated words by removing repeated characters with three or more characters. For example, the word "bagusss" is replaced with "bagus".
- Whitespace tokenization by using stop-word removal using a combined English (NLTK) and Bahasa Malaysia stop-word list.

- Negative words such as “tidak”, “bukan”, “no” and “not” are not removed because they invert sentiment.
- English tokens are lemmatized using NLTK's WordNetLemmatizer while Malay words are passed through without doing any changes.

Filtering and Deduplication

Duplicated reviews and reviews that are too short which are reviews with less than three tokens were removed. There are a total of 558 duplicated review texts and 560 reviews containing less than three tokens. The final dataset set is then saved into cleaned_data.csv.

Feature Engineering

Six features were derived so that it can be used for analysis. The derived columns are namely original word count, cleaned word count, contains emoji, exclamation mark count, question mark count and character count. The final dataset has a total of 15 columns in total including the original columns mentioned before.

2.3 Sentiment Labelling

A pre-trained model called *cardiffnlp/twitter-roberta-base-sentiment-latest* was used to label the review sentiments. The model was trained using ~124 million tweets and was fine tuned for sentiment analysis. It is well suited for informal and casual text such as app reviews and can classify the reviews with either positive, negative or neutral. The chosen model is justified because ~98% of the reviews are in English which will not affect the model performance as much.

The result of the sentiment labelling showed an imbalance of classes. Where ~78% percent of the reviews are classified as negative. The reviews classified as positive are around ~16% followed by the neutral class of around only ~6%. The mean prediction confidence value is 0.83 where only ~8% of the reviews are below 0.6 confidence rate which indicates the model is generally confident on the data it labels. The result can be summarized in Table 2.2 and Figure 2.1 below.

Table 2.2 Sentiment Label Distribution

Sentiment	Count	Percentage
Negative	13,068	77.4%
Neutral	1,074	6.4%
Positive	2,740	16.2%
Total	16,882	100%

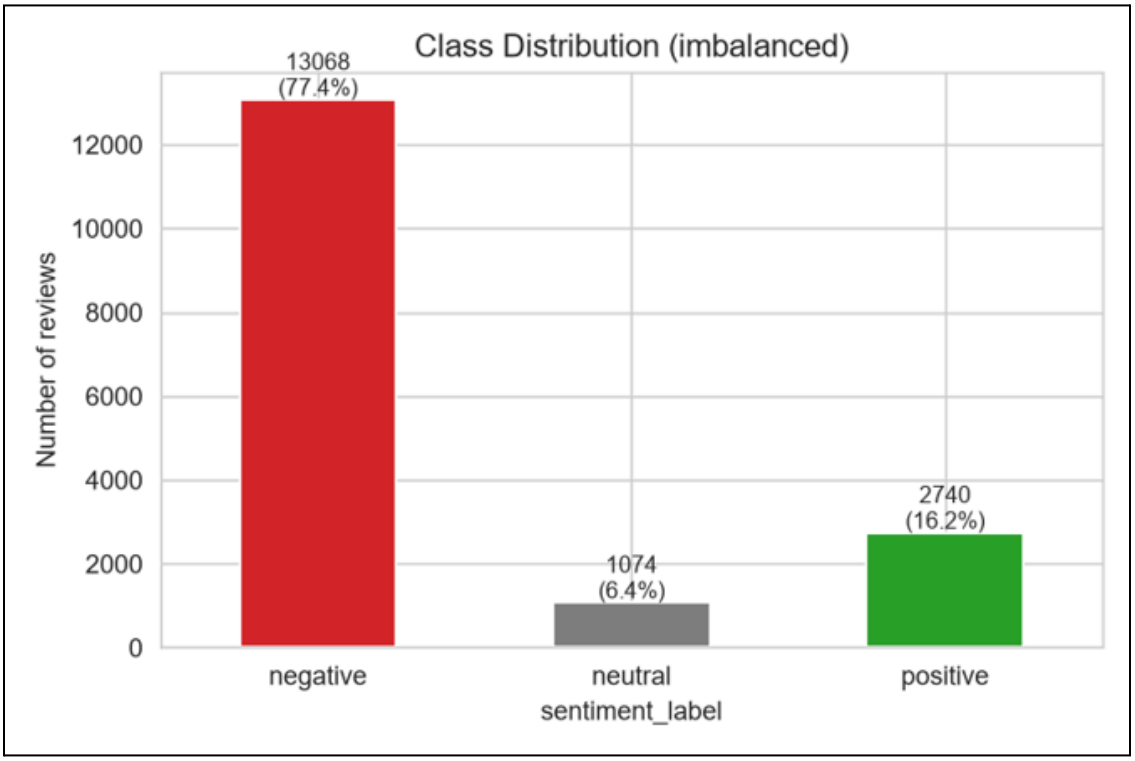

Figure 2.1: Sentiment Class Distribution Graph

3.0 Sentiment Model Development

3.1 Data Splitting

Table 3.1 below shows the dataset was split into training, validation and test subsets using a stratified 70/20/10 ratio. It preserves the class proportions for each sentiment so that all the sentiments are the same ratio for each subset.

Table 3.1: Data Splitting Ratio

Subset	Records	Proportion	Purpose
Training	11,816	70%	Fit model parameters
Validation	3,377	20%	Model selection
Test	1,689	10%	Final unbiased evaluation

3.2 Handling Class Imbalance

During training, balanced class weights were used to address the imbalance without producing artificial data. Because of this, the model does not only ignore errors on the rare classes (positive and neutral), treating them as more expensive. The weights of each classes are shown in Table 3.2 below:

Table 3.2: Weight Distribution for Handling Class Imbalance

Class	Train count	Balanced weight
Negative	9,147	0.43
Positive	1,918	2.05
Neutral	751	5.25

Macro-averaged F1 was used as the main metric as it treated every class equally instead of just using accuracy to evaluate the model.

3.3 Models

There are two models used in this project for training and comparison from one from these categories shown in Table 3.3.

Table 3.3: Model Category and Model Used

Category	Model Used
Machine Learning	**Logistic Regression:** A classical model trained on TF-IDF text features, with balanced class weights.
Deep Learning	**Bidirectional LSTM:** A neural network with a word-embedding layer and a bidirectional LSTM, trained in PyTorch with the same balanced class weights.

3.4 Evaluation and Comparison

Both models were evaluated on the test set using accuracy, precision, recall, F1 and confusion matrices. The results are shown in Table 3.4 and Figure 3.1 below.

Table 3.4: Model Evaluation Result

Model	Accuracy	Macro-F1	Weighted-F1
Logistic Regression	0.885	0.726	0.891
LSTM	0.843	0.689	0.856

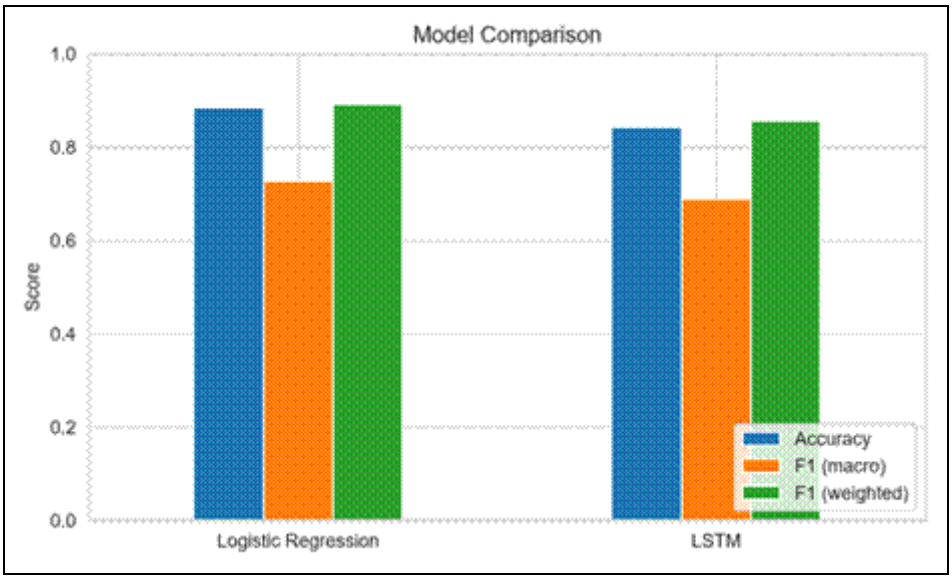

Figure 3.1: Model comparison on the test set.

Per-class results on the test set (precision / recall / F1):

Class	Logistic Regression	LSTM
Negative	0.95 / 0.93 / 0.94	0.95 / 0.86 / 0.90
Neutral	0.33 / 0.43 / 0.37	0.27 / 0.44 / 0.33
Positive	0.88 / 0.87 / 0.87	0.76 / 0.93 / 0.83

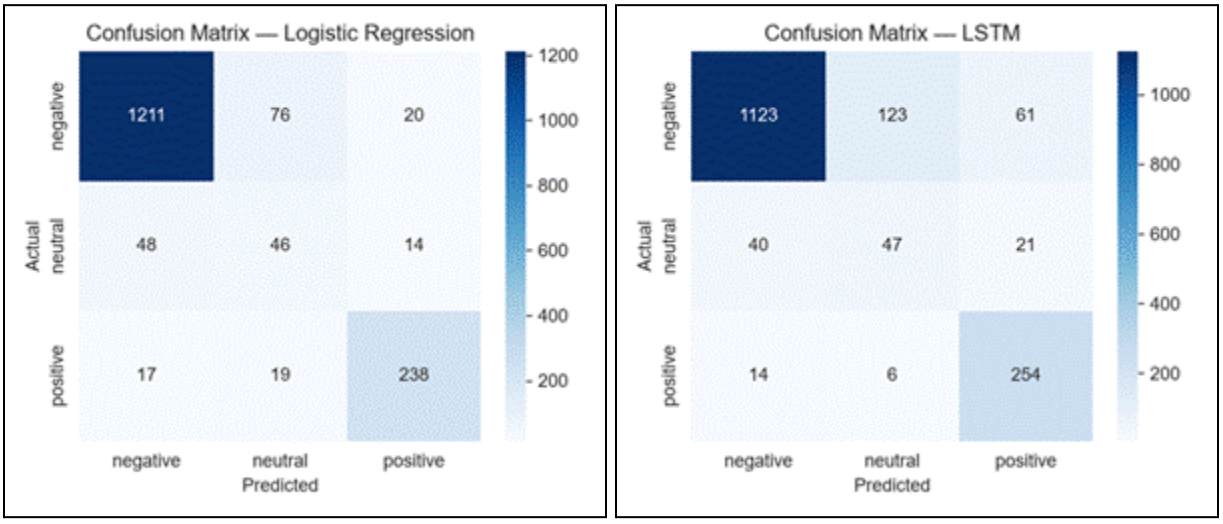

Figure 3.2: Confusion matrices of Logistic Regression (left) and LSTM (right).

Logistic Regression performed best because of higher macro-F1 and accuracy also is lighter to run, so it was chosen as the model deployed in the pipeline. The neutral class is the hardest for both models because it is rare. The class-weighting clearly worked where both models still detect the positive and neutral classes well instead of only predicting negative.

4.0 Apache System Architecture

The system is divided into two stages. An offline stage where data is acquired, prepared and used to train the models and the real-time streaming stage which runs in a Docker environment that ingests, classifies, stores and visualizes the review. The overall architecture is shown in Figure 4.1.

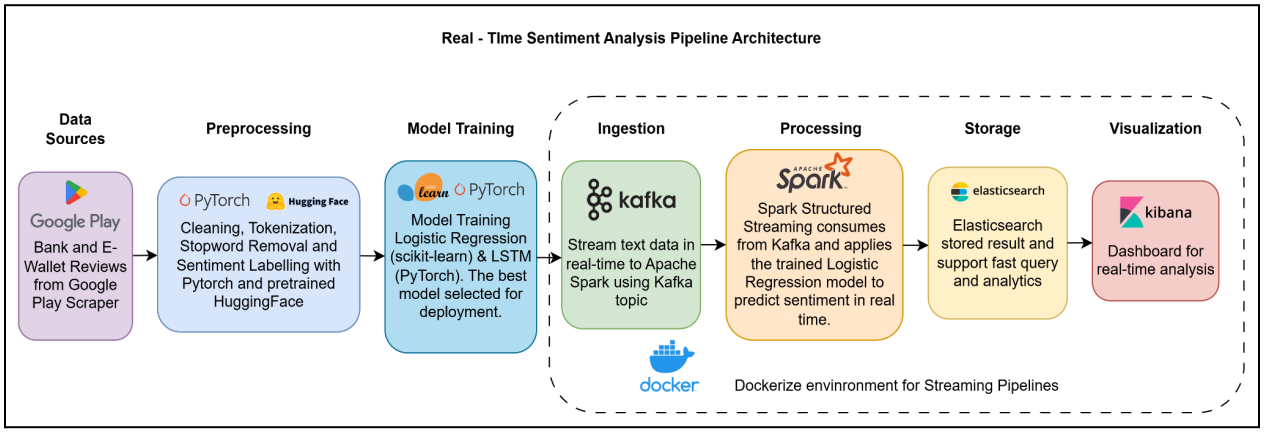

Figure 4.1: Real-Time Analysis Pipeline.

The pipeline is made up of the following phases shown in Table 4.1

Table 4.1: Pipeline Phases, Tools and Description

Phase	Tools	Description
Data Sources	Google Play Scraper	Bank and e-wallet app reviews are collected from the Google Play Store.

Preprocessing	NLP libraries, HuggingFace + PyTorch	Text is cleaned, tokenized, stop-words removed and lemmatized; sentiment labels are generated using a pretrained HuggingFace/PyTorch model.
Model Training	scikit-learn, PyTorch	Logistic Regression and LSTM are trained and compared; the best model (Logistic Regression) is selected for deployment.
Ingestion	Apache Kafka	Reviews are streamed in real time into a Kafka topic.
Processing	Apache Spark	Spark Structured Streaming consumes from Kafka and applies the trained model to predict sentiment.
Storage	Elasticsearch	Classified results are stored and indexed for fast querying.
Visualization	Kibana	An interactive dashboard presents the results for real-time analysis.

4.1 Data Preparation and Model Training

The data is first scraped using the *google-play-scraper* Python library of popular e-wallet and banking applications reviews. The raw reviews are then cleaned, processed, tokenized, remove stopwords and lemmatized before the sentiment can be labelled. The sentiment labelling phase uses a pre-trained model by Hugging Face Transformer to label the sentiment classes. Then the labelled reviews are used to train two classification models namely Logistic Regression and LSTM. The models are then compared where the model that performs best are saved and will later be loaded by the streaming. The model that is used for the pipeline is the Logistic Regression model.

4.2 Real-Time Streaming

The real-time streaming pipeline is run in a container with Docker Compose which contains Apache Kafka, Apache Spark, Elasticsearch and Kibana. Apache Kafka is used for the ingestion stage where it acts as a message broker. A producer sends each review as a message to Kafka topic that simulates live, continuous stream. Apache Spark is the processing unit where

Spark Structures Streaming reads the messages from Kafka as small micro-batches and loads the trained logistic Regression model to predict the sentiment and confidence score of each review. The results are then stored into the Elasticsearch index which is optimized for fast searching aggregation. Kibana is then used to read from Elastisearch and displays the result as an interactive dashboard for real-time analysis.

5.0 Analysis and Result

5.1 Sentiment Distribution

There are 16,882 reviews in total streamed through the pipeline and classified successfully. The Kibana dashboard shows the results through cards which are total review and average confidence. The dashboard also has a sentiment pie chart, a word cloud of frequent terms in the reviews, each app sentiment breakdown and a sentiment over time chart.

Table 5.1: Sentiment Distribution in the Pipeline

Sentiment	Count	Percentage
Negative	12,588	74.6%
Positive	2,867	17.0%
Neutral	1,427	8.5%

5.2 Dashboard

The Kibana dashboard provides a summary on all 16,882 classified reviews using six different charts. Two KPI charts show the number of total reviews and the average confidence rate of the developed model. Then a pie chart shows the overall split of the reviews sentiment label where the negative sentiment represents the highest count followed by positive and neutral sentiment. A word cloud chart highlights the most frequent terms for the overall reviews which gives insight on what most users complain about about the app. A stacked bar chart was used to compare the nine apps which shows the distribution of the sentiments of each label of each app. The chart shows that Boost and CIMB have more negative sentiment to total review ratio while Shopee receives the best sentiment showing a more balanced sentiment ratio. The line chart on the other hand, shows the review volume growing ~167 in 2018 to ~3000 reviews by 2024 t0 2026. The dashboard provides a view on the number of feedbacks, how the customer feels, what they talk about, the performance of each app and the sentiment trends over time.

Figure 5.1: Dashboard in Kibana.

6.0 Optimization and Comparison

6.1 Model Comparison

As shown in Section 3, Logistic Regression outperformed the LSTM model and was chosen to be deployed in the streaming pipeline. It is also much faster to run, which suits the real world pipelines.

6.2 Batch vs Streaming Comparison

The same model, Logistic Regression was run streaming over 16882 reviews in two ways which are batch and streaming mode (through Kafka and Spark). Table 6.1 and Figure 6.1 below shows the result of batch and streaming performances.

Table 6.1: Comparison of Batch and Streaming Performance

Metric	**Batch**	**Streaming**
Records processed	16,882	16,882
Total time (s)	0.56	6.02
Throughput (rec/s)	30,188	2,807
Accuracy	0.94	0.95
Avg latency (s)	-	3.83

Batch mode processes everything at once, making it significantly faster. This is also reflected in the lower total time (s), higher throughput and no average latency(s). Streaming mode is slower and adds some delay per record, but it can analyze data continuously as it enters, which is the purpose of a real-time system. However, switching to streaming does not lower quality for this case because the accuracy is nearly identical in both modes where batch and streaming mode has an accuracy reading of 0.94 and 0.95 respectively which is not a significant difference.

7.0 Conclusion and Future Work

For Malaysian banking and e-wallet app reviews, this project developed a complete, functional real-time sentiment analysis system. The best model (Logistic Regression) was implemented in this pipeline after two models were trained and compared and the class-imbalance issue was addressed with balanced class weights. Batch and streaming modes were compared, and the system was thoroughly tested on all 16,882 reviews.

In the future, the pipeline could be connected to a live data source rather than a replayed dataset, a transformer model like DistilBERT could be tried, the neutral class could be improved with further data and the models could be further tuned.

8. References

1. Apache Kafka Documentation. https://kafka.apache.org/documentation/
2. Apache Spark Structured Streaming Programming Guide. https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html
3. Elasticsearch Guide. https://www.elastic.co/guide/
4. Kibana Guide. https://www.elastic.co/guide/en/kibana/current/
5. scikit-learn: Machine Learning in Python. https://scikit-learn.org/
6. PyTorch Documentation. https://pytorch.org/docs/
7. Sentiment Labelling (cardiffnlp/twitter-roberta-base-sentiment-latest) Dcumentation: https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest